



Universidad Veracruzana

Proyecto

Laundries

Documentación del proyecto

presenta:

Gabriel Armas Viveros

Mauricio Salas García

Thomas Alejandro Toro Herrera

12 de enero del 2025

Índice

Introducción:	4
Descripción del problema:	4
Objetivo del proyecto:	4
Requerimientos:	5
Requisitos funcionales (RF):	5
Requisitos no funcionales (RNF):	7
Diseño:	8
Diagrama de casos de uso:	8
Diagrama de secuencia:	12
CU-01: Iniciar sesión:	12
CU-02: Cerrar sesión:	13
CU-03: Ver cliente:	13
CU-04: Buscar cliente:	14
CU-05: Editar cliente:	14
CU-06: Ver servicios activos:	15
CU-07: Realizar venta:	15
CU-08: Descargar comprobante:	16
CU-09: Modificar orden:	16
CU-10: Cancelar orden:	17
CU-11: Ver ventas activas:	17
CU-12: Ver ticket:	18
CU-13: Realizar corte de caja:	18
CU-14: Gestionar empleados sucursal:	19
CU-15: Ver ventas activas de la sucursal:	20
CU-16: Ver detalle de una venta activa:	20
CU-17: Obtener código de cancelación:	21
CU-19: Ver sucursales:	22
CU-20: Administrar gerentes:	23
CU-21: Ver todos los empleados:	24
CU-22: Gestionar servicios de lavandería:	25
CU-23: Generar reporte global:	26
CU-24: Descargar reporte:	26
Diagrama de despliegue:	27
Diagrama entidad relación:	29
Diagrama de orders-api:	29
Diagrama de branches-api:	30
- Diagrama de empleados-api:	30
- Diagrama de sucursales-api:	30
- Diagrama de corte-caja-api:	31
Diagrama de auth-api:	31

Casos de uso:.....	31
Historias de Usuario Generales: Employee, Manager y Admin.....	32
CU-01: Iniciar sesión.....	32
CU-02: Cerrar sesión.....	32
Historias de Usuario: Employee (flujo diario).....	33
CU-03: Ver cliente.....	33
CU-04: Buscar cliente.....	33
CU-05: Editar cliente.....	33
CU-06: Ver servicios activos.....	33
CU-07: Realizar venta.....	33
CU-08: Descargar comprobante.....	34
CU-09: Modificar orden.....	34
CU-10: Cancelar orden.....	34
CU-11: Ver ventas activas.....	34
CU-12: Ver ticket.....	34
CU-13: Realizar corte de caja.....	35
Historias de Usuario: Manager (Supervisa y Controla).....	35
CU-14: Gestionar empleados sucursal.....	35
CU-15: Ver ventas activas de la sucursal.....	35
CU-16: Ver detalle de una venta activa.....	35
CU-17: Obtener código de cancelación.....	36
CU-18: Generar reportes de sucursal.....	36
Historias de Usuario: Admin (General).....	36
CU-19: Ver sucursales.....	36
CU-20: Administrar gerentes.....	36
CU-21: Ver todos los empleados.....	37
CU-22: Gestionar servicios de lavandería.....	37
CU-23: Generar reporte global.....	37
CU-24: Descargar reporte.....	37
Construcción:.....	38
Selección justificada de la arquitectura:.....	38
Selección justificada de las tecnologías:.....	40
Pruebas:.....	41
Conclusiones:.....	45

Proyecto lavandería

Introducción:

Descripción del problema:

Actualmente, muchas cadenas de lavandería continúan operando mediante procesos manuales o semi-manuales para la gestión de clientes, pedidos y control de ventas. Esta forma de trabajo provoca múltiples problemáticas operativas, como la pérdida de pedidos, errores en el registro de prendas, dificultad para rastrear el estado de las órdenes, y falta de control en los cortes de caja y reportes financieros.

Además, la ausencia de un sistema tecnológico centralizado impide a los gerentes y administradores contar con información en tiempo real sobre el estado de las operaciones, el desempeño del personal y los ingresos por sucursal. Esto genera dependencia excesiva del factor humano, aumenta la probabilidad de errores y dificulta la toma de decisiones estratégicas basadas en datos confiables.

Con ello, se ha identificado la necesidad de desarrollar un sistema de información para lavanderías que permita automatizar los procesos clave, mejorar la trazabilidad de las órdenes, reforzar la seguridad de la información y optimizar la administración de sucursales, empleados y servicios.

Objetivo del proyecto:

Desarrollar un sistema web de gestión para cadenas de lavandería que permita automatizar y centralizar los procesos operativos, administrativos y de control, facilitando el registro, seguimiento y

administración de clientes, órdenes, ventas, empleados y reportes, con base en roles definidos (Empleado, Gerente y Administrador).

Requerimientos:

Requisitos funcionales (RF).

ID	Descripción
RF-01	El sistema debe permitir registrar la recepción de una o varias prendas por cliente.
RF-02	El sistema debe generar un ticket por cada recepción de prendas como comprobante.
RF-03	El sistema debe permitir asociar las prendas recibidas a un tipo de servicio ofrecido.
RF-04	El sistema debe calcular y asignar un costo según el tipo de servicio seleccionado.
RF-05	El sistema debe permitir realizar el cobro correspondiente al servicio contratado.
RF-06	El sistema debe registrar la entrega de las prendas al cliente al momento de su devolución.

RF-07	El sistema debe permitir la operación de múltiples sucursales dentro de la cadena.
RF-08	El sistema debe permitir realizar el corte de caja al final del día por cada sucursal.
RF-09	El sistema debe generar reportes de operaciones, servicios y cobros.
RF-10	El sistema debe generar gráficos a partir de la información registrada.

Requisitos no funcionales (RNF).

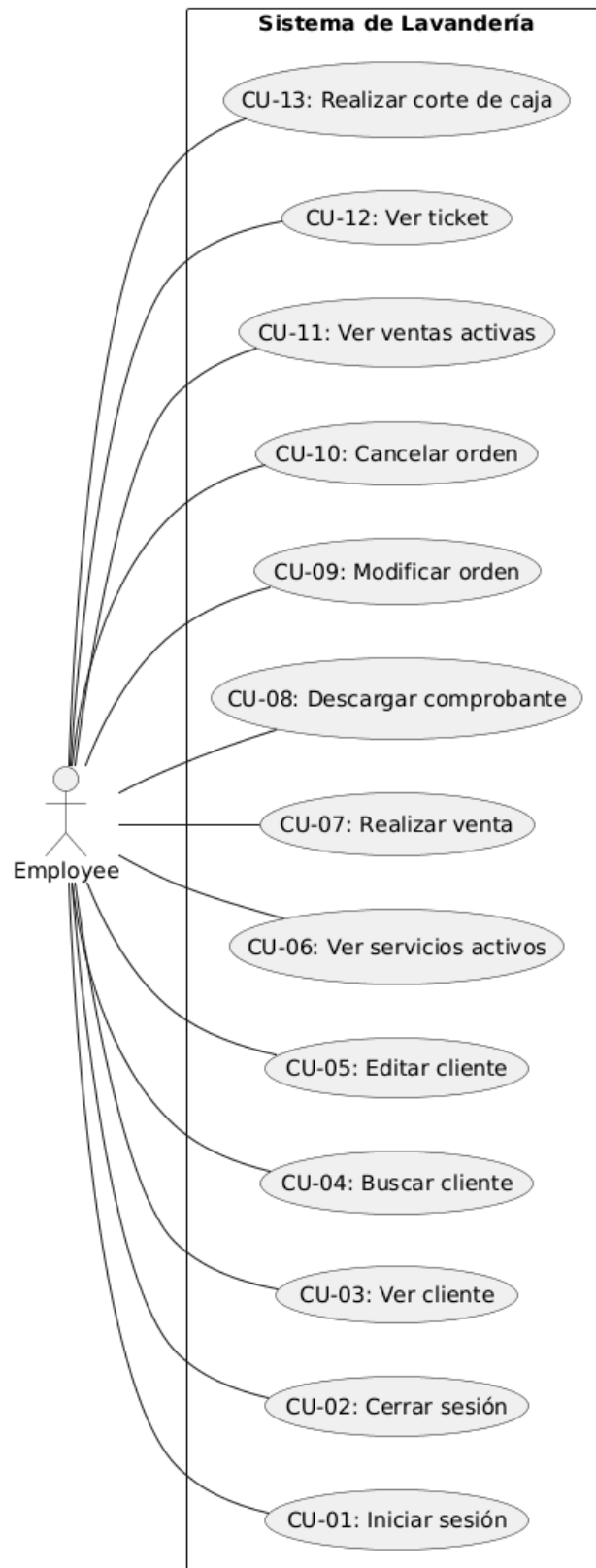
ID	Descripción
RNF-01	El sistema debe soportar la operación simultánea de múltiples sucursales.
RNF-02	El sistema debe garantizar la integridad y consistencia de la información.
RNF-03	El sistema debe contar con una interfaz clara y comprensible para el usuario.
RNF-04	El sistema debe ofrecer tiempos de respuesta adecuados en sus operaciones principales.
RNF-05	El sistema debe ser escalable para permitir la incorporación de nuevas sucursales o servicios.

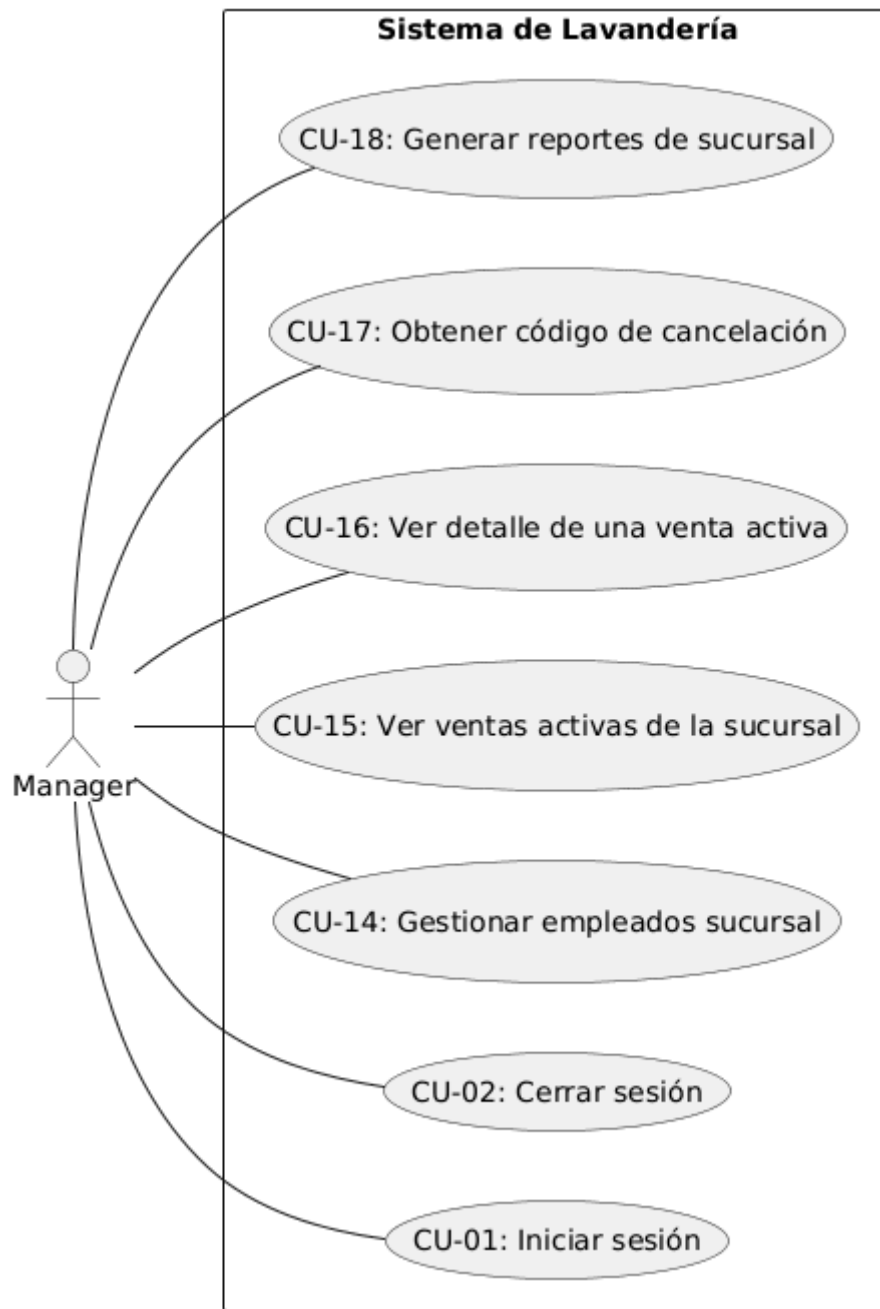
Diseño:

Diagrama de casos de uso:

El diagrama de casos de uso constituye una importante forma de documentar el software. Este diagrama representa de forma visual cada usuario con las acciones que realiza dentro del sistema, para así comprender fácilmente sus funcionalidades y necesidades.

A continuación se presentan cada uno de los diagramas divididos según el usuario.





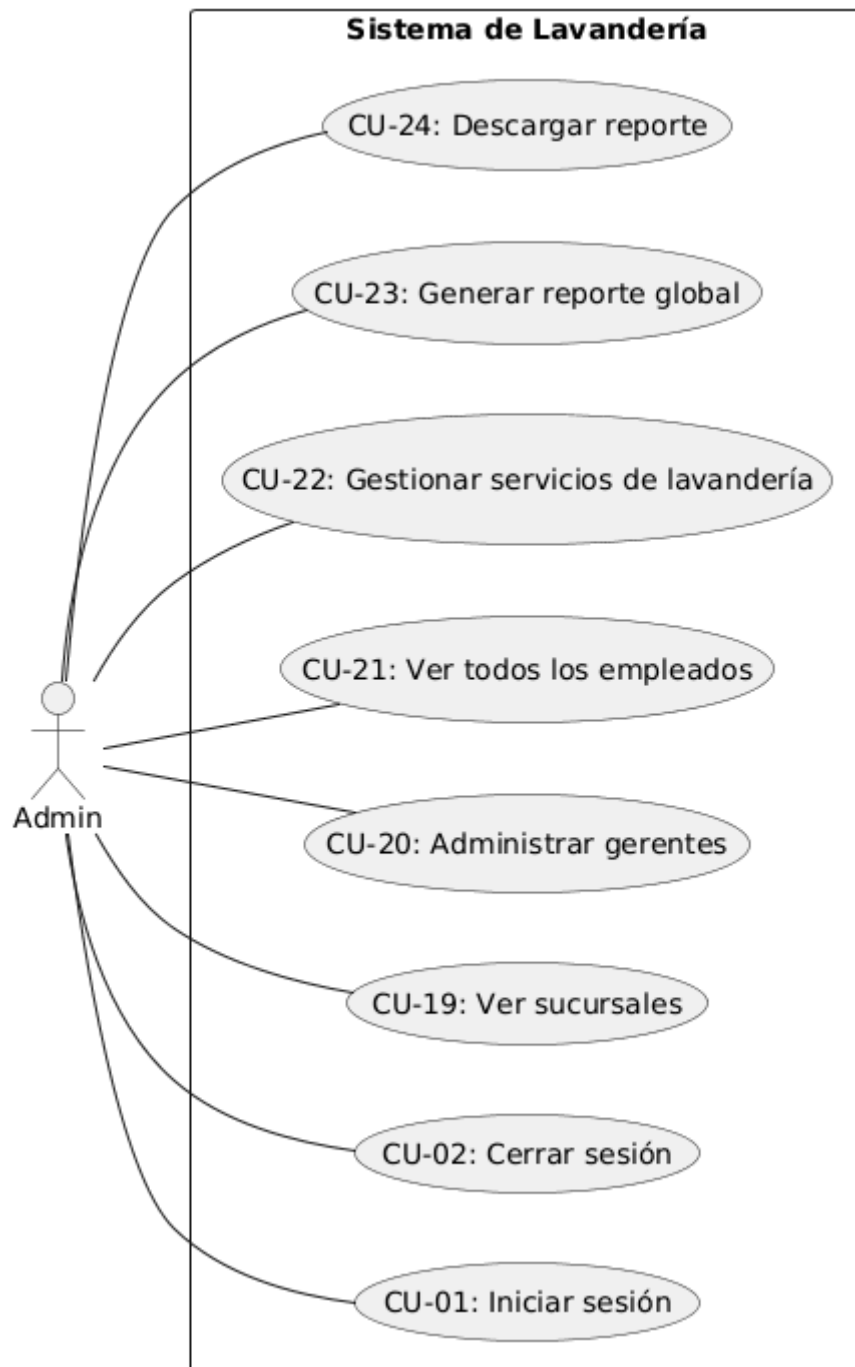
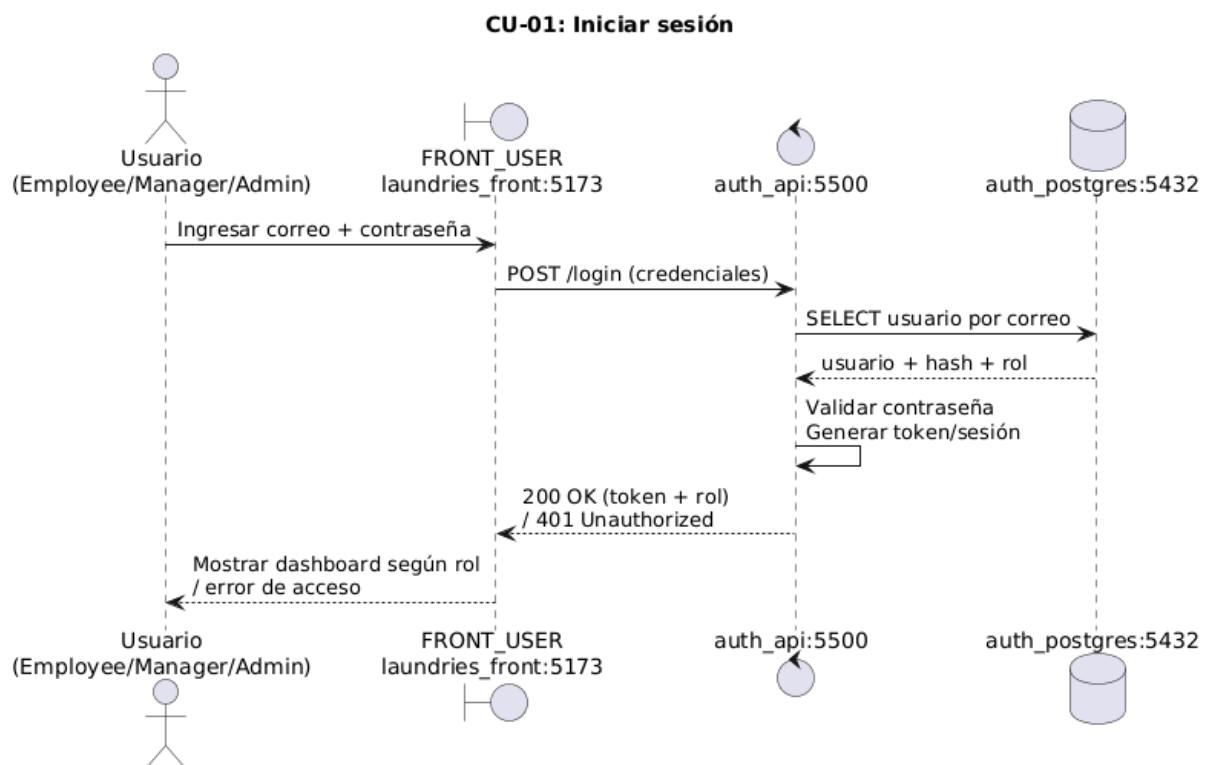


Diagrama de secuencia:

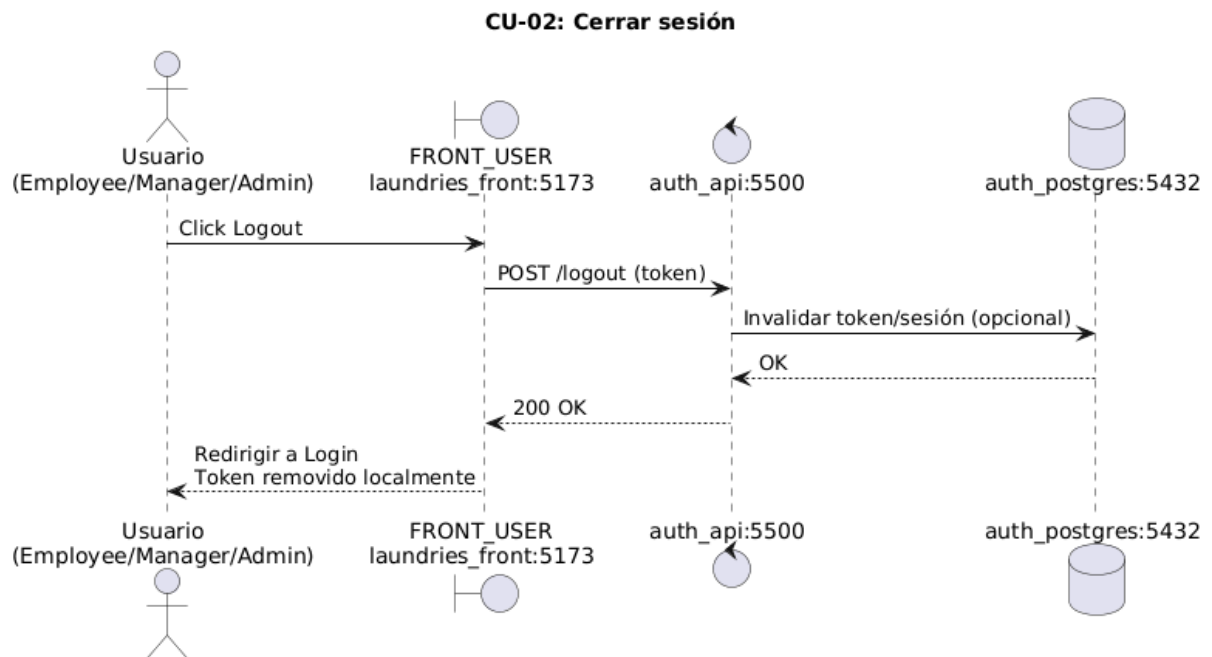
Los diagramas de secuencia representan las operaciones que realizan los usuarios y el sistema para llevar a cabo un caso de uso. Se ha decidido utilizarlos para el proyecto por su buena manera de representar las llamadas entre partes del sistema y así comprender su funcionalidad interna.

A continuación se presentan separados cada uno según su caso de uso.

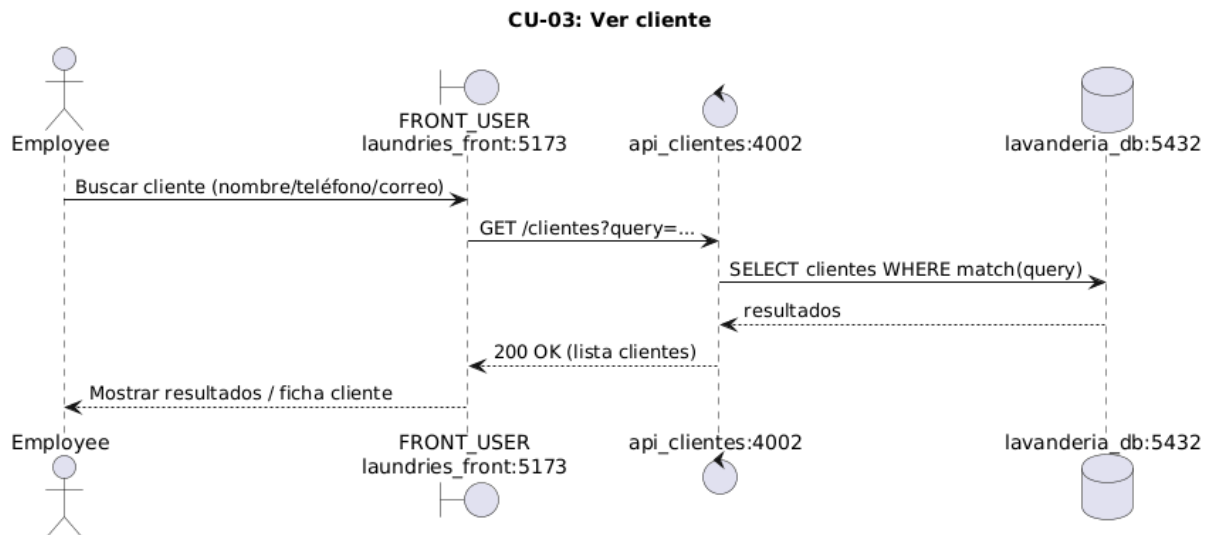
CU-01: Iniciar sesión.



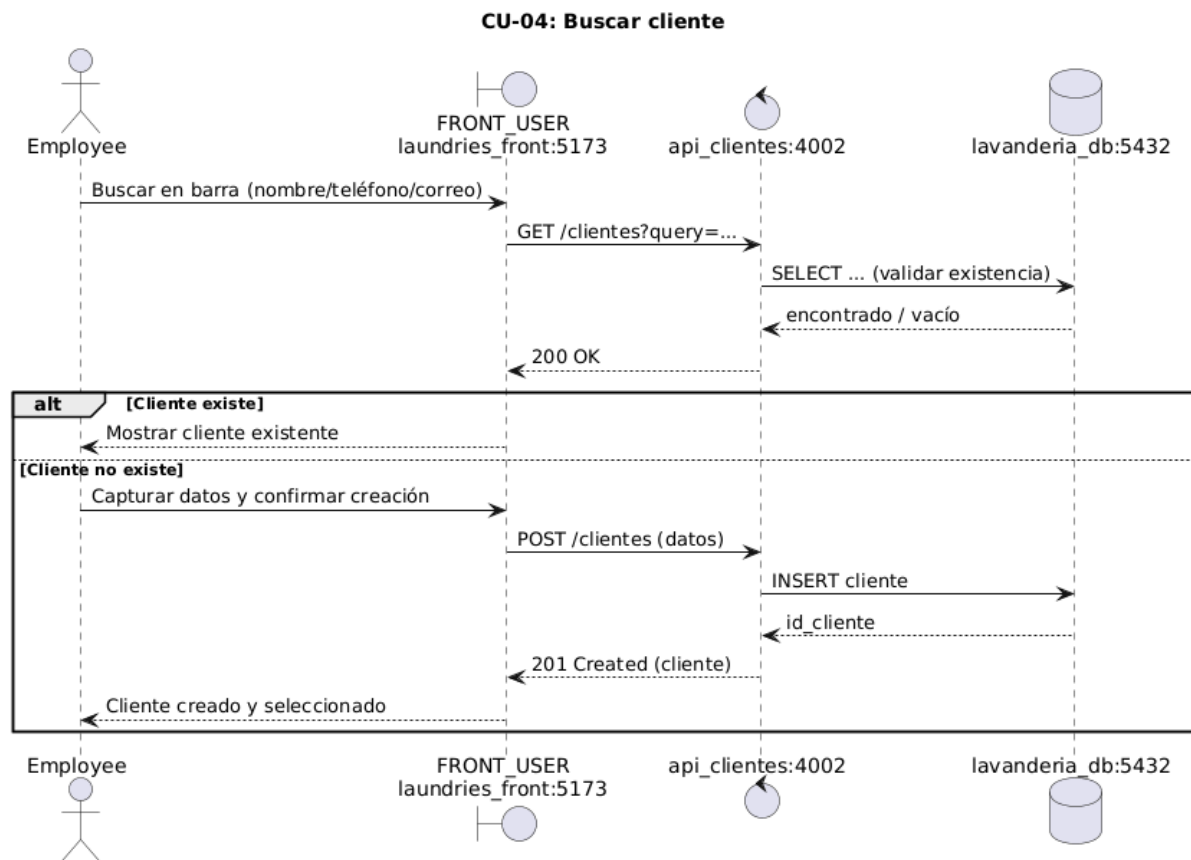
CU-02: Cerrar sesión.



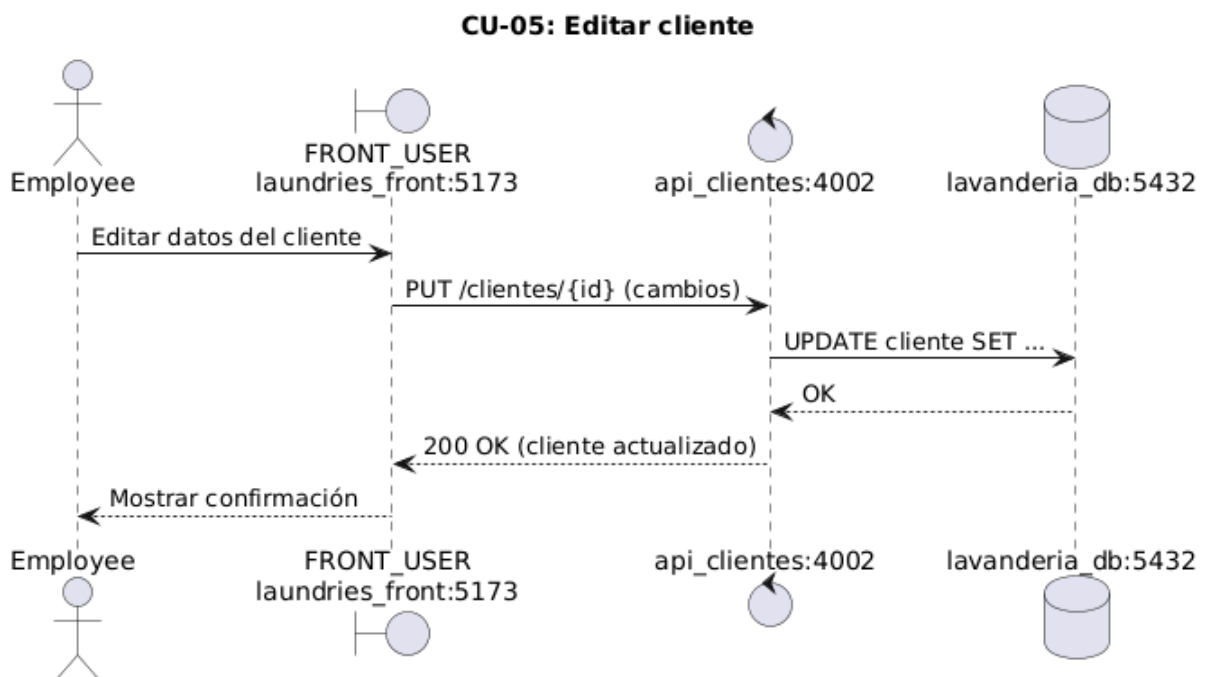
CU-03: Ver cliente.



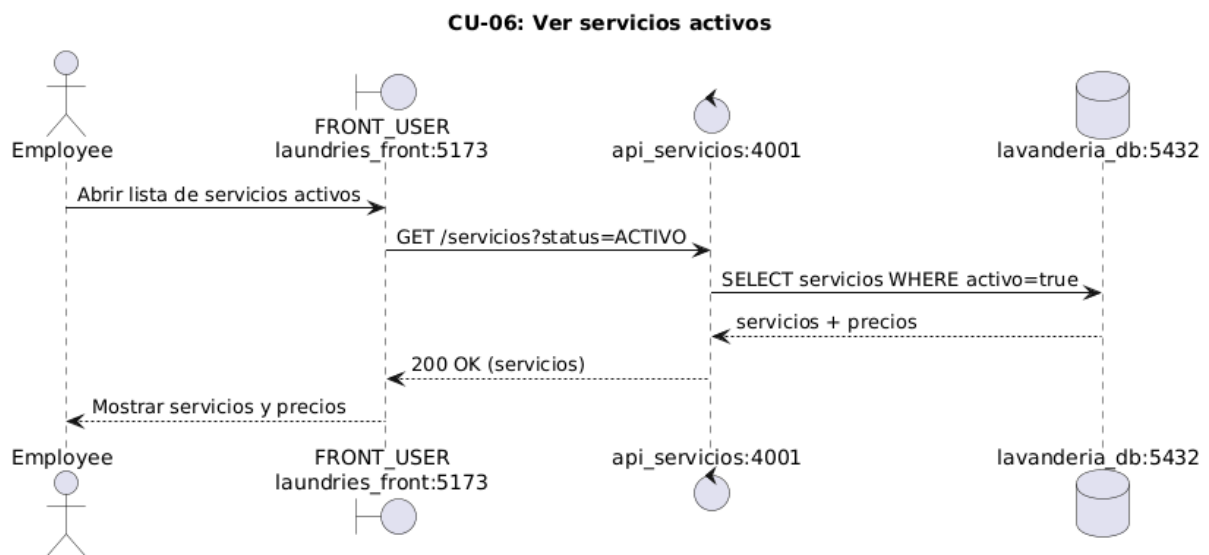
CU-04: Buscar cliente.



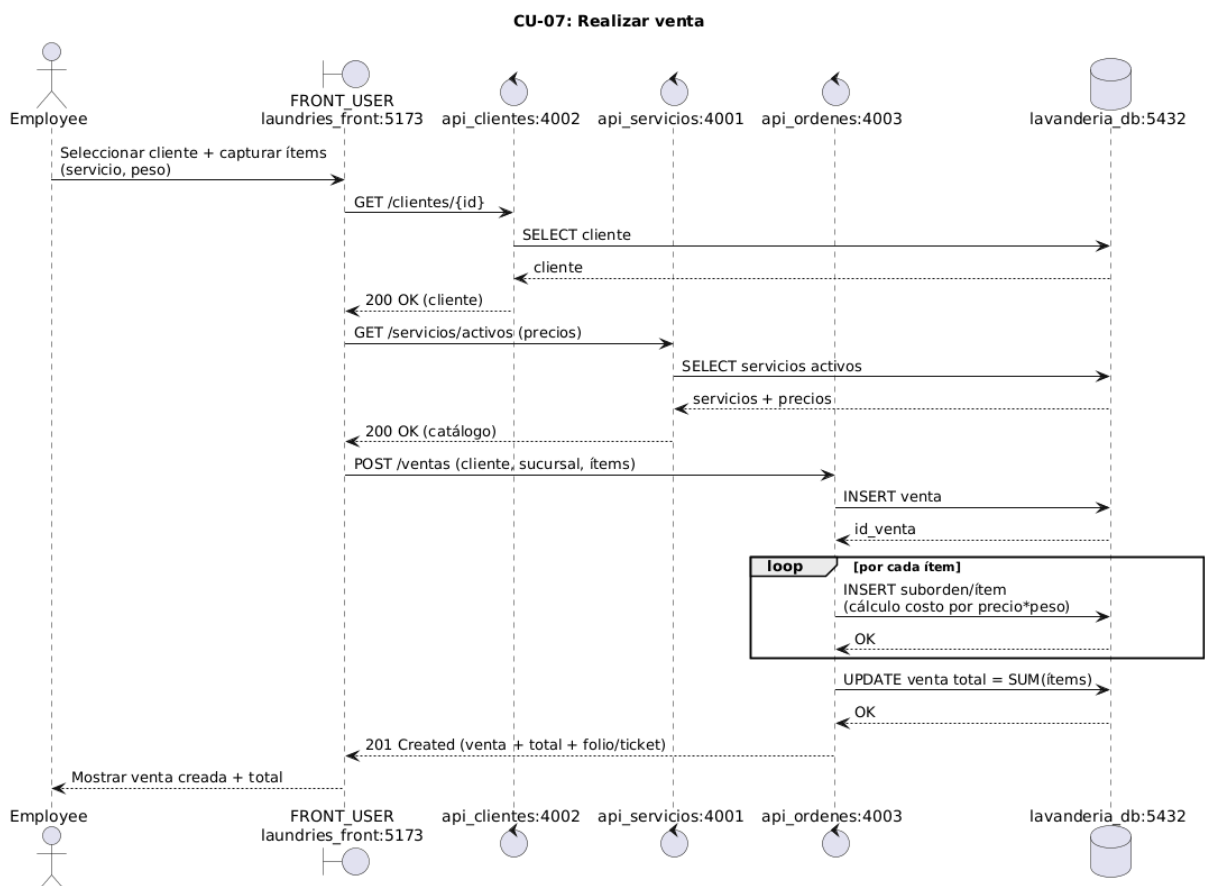
CU-05: Editar cliente



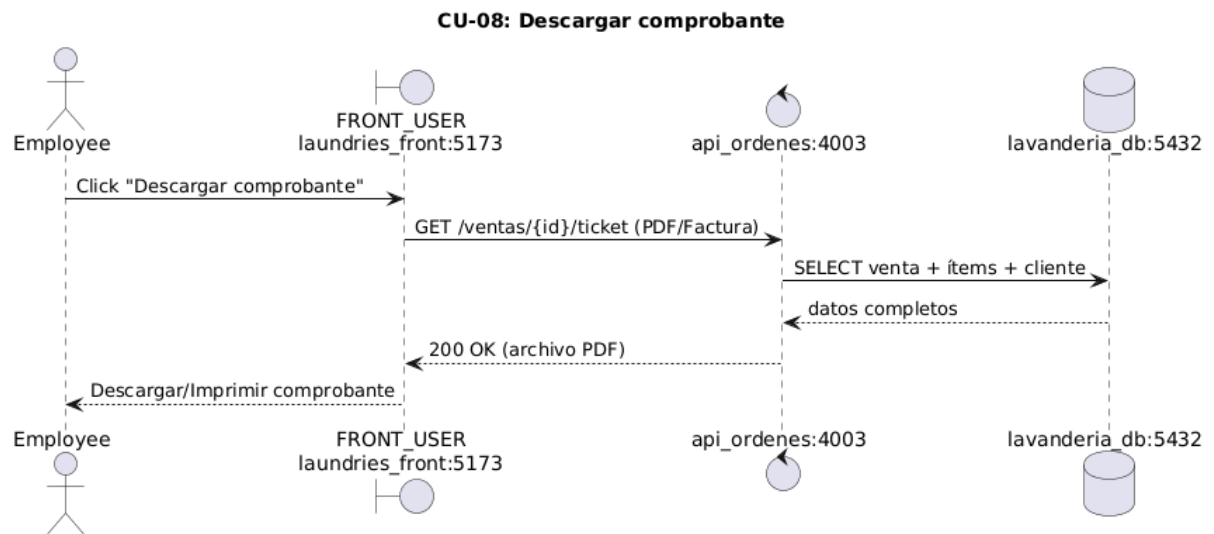
CU-06: Ver servicios activos.



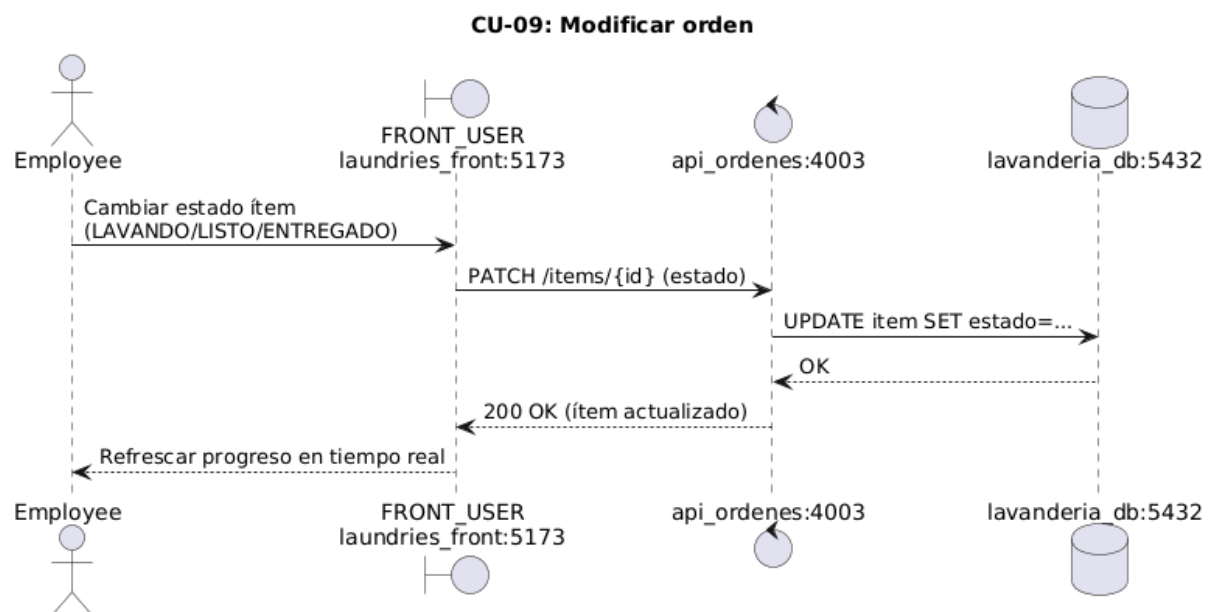
CU-07: Realizar venta



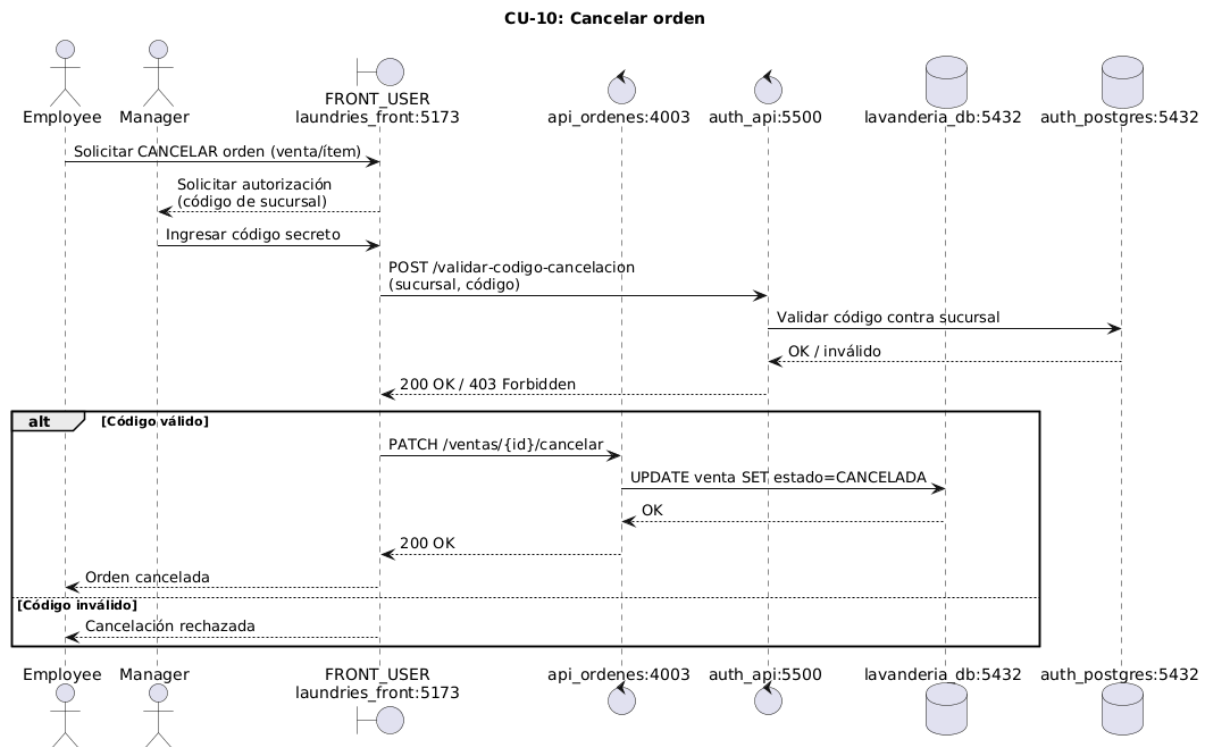
CU-08: Descargar comprobante



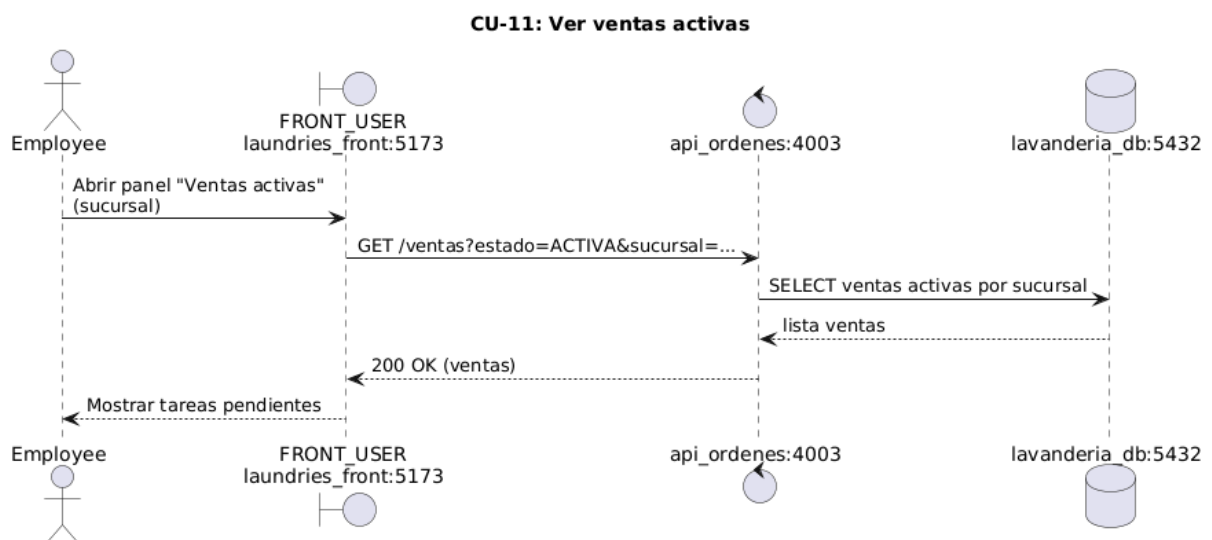
CU-09: Modificar orden



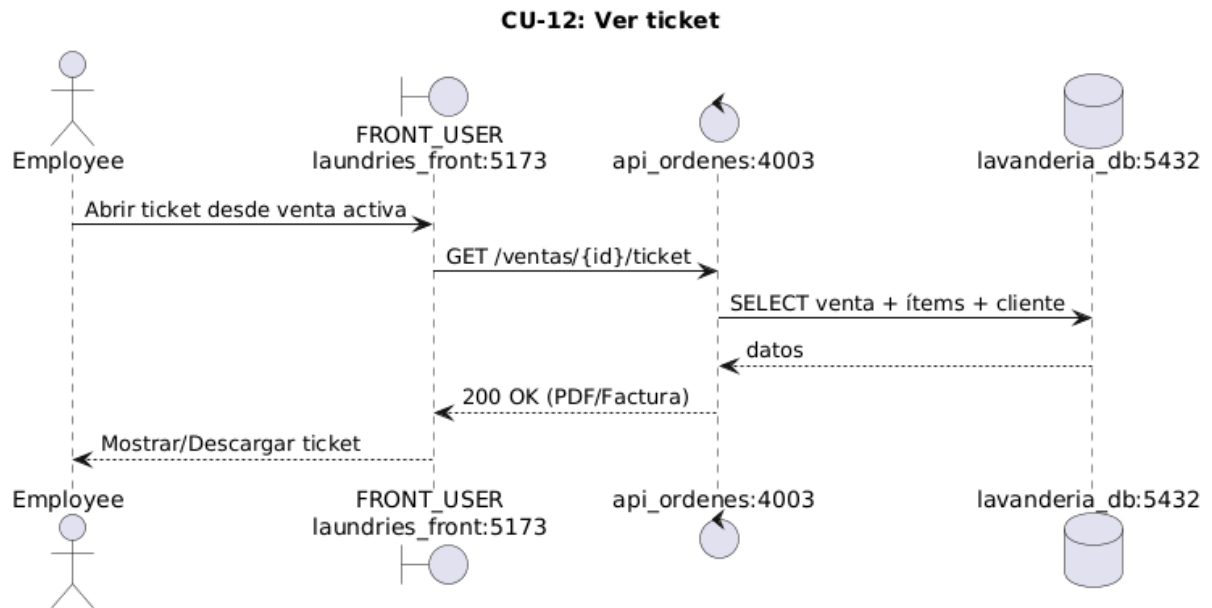
CU-10: Cancelar orden



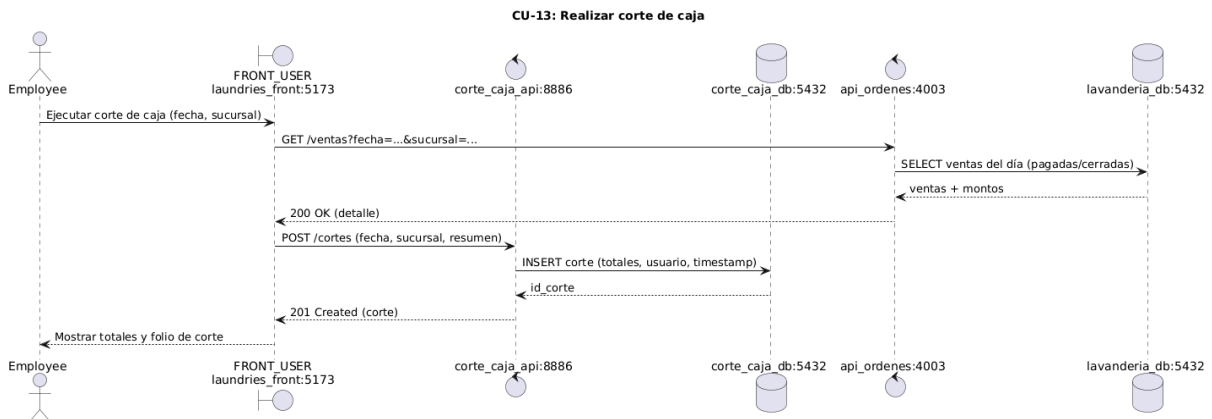
CU-11: Ver ventas activas.



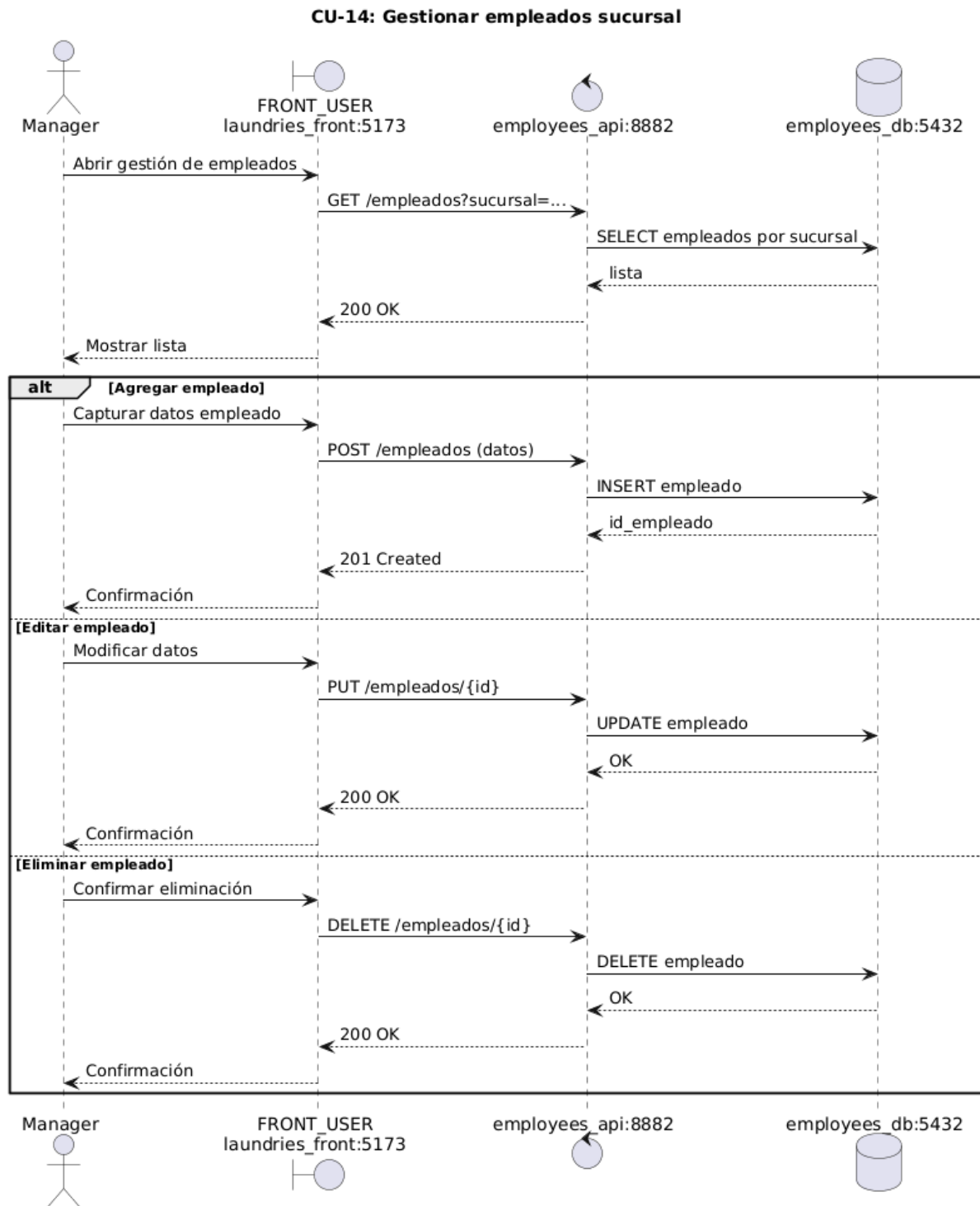
CU-12: Ver ticket



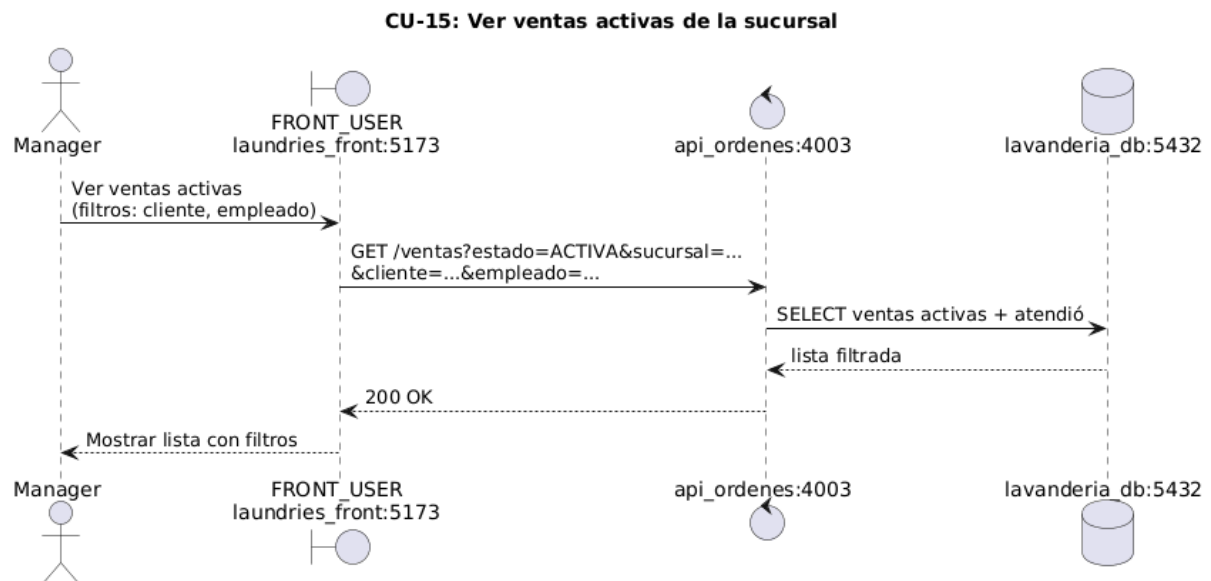
CU-13: Realizar corte de caja.



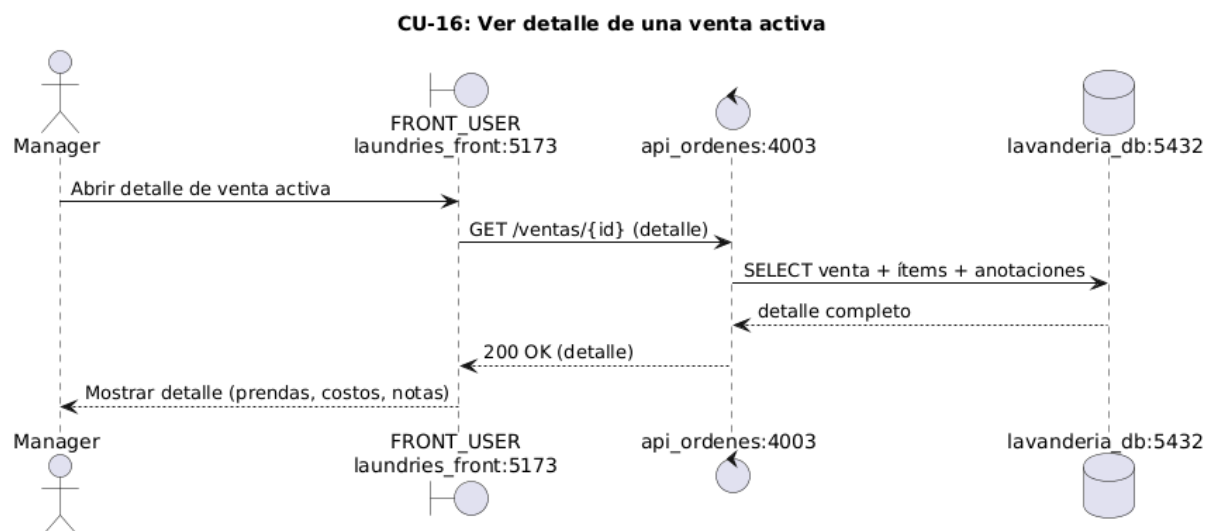
CU-14: Gestionar empleados sucursal.



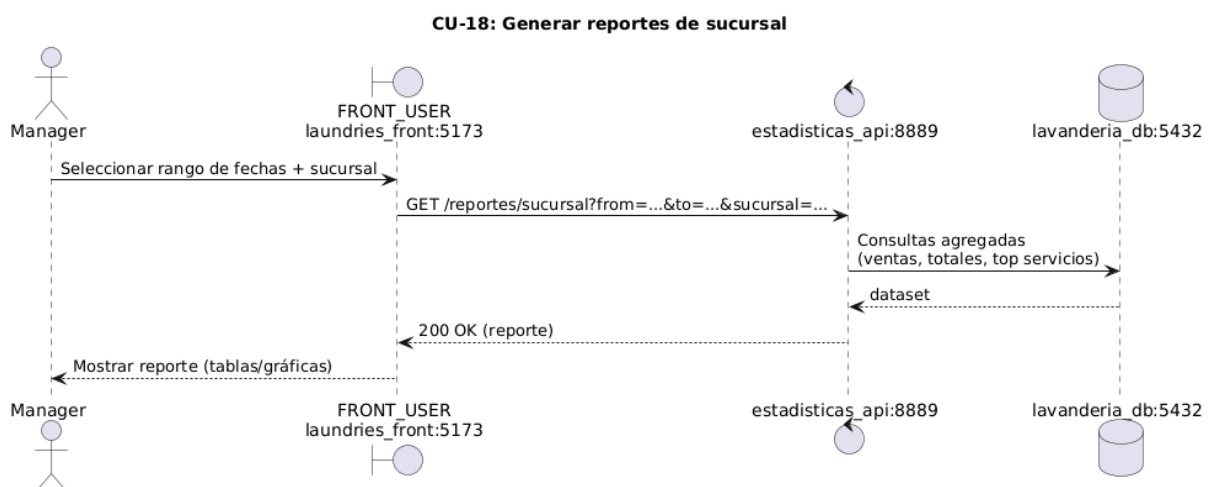
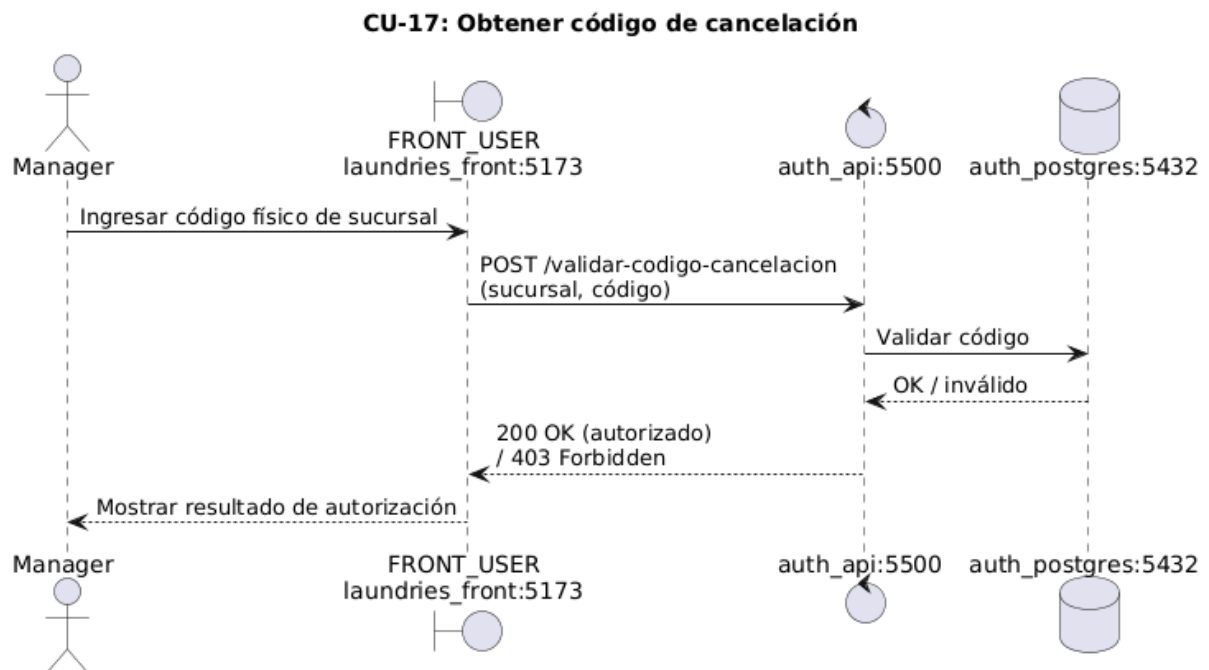
CU-15: Ver ventas activas de la sucursal.



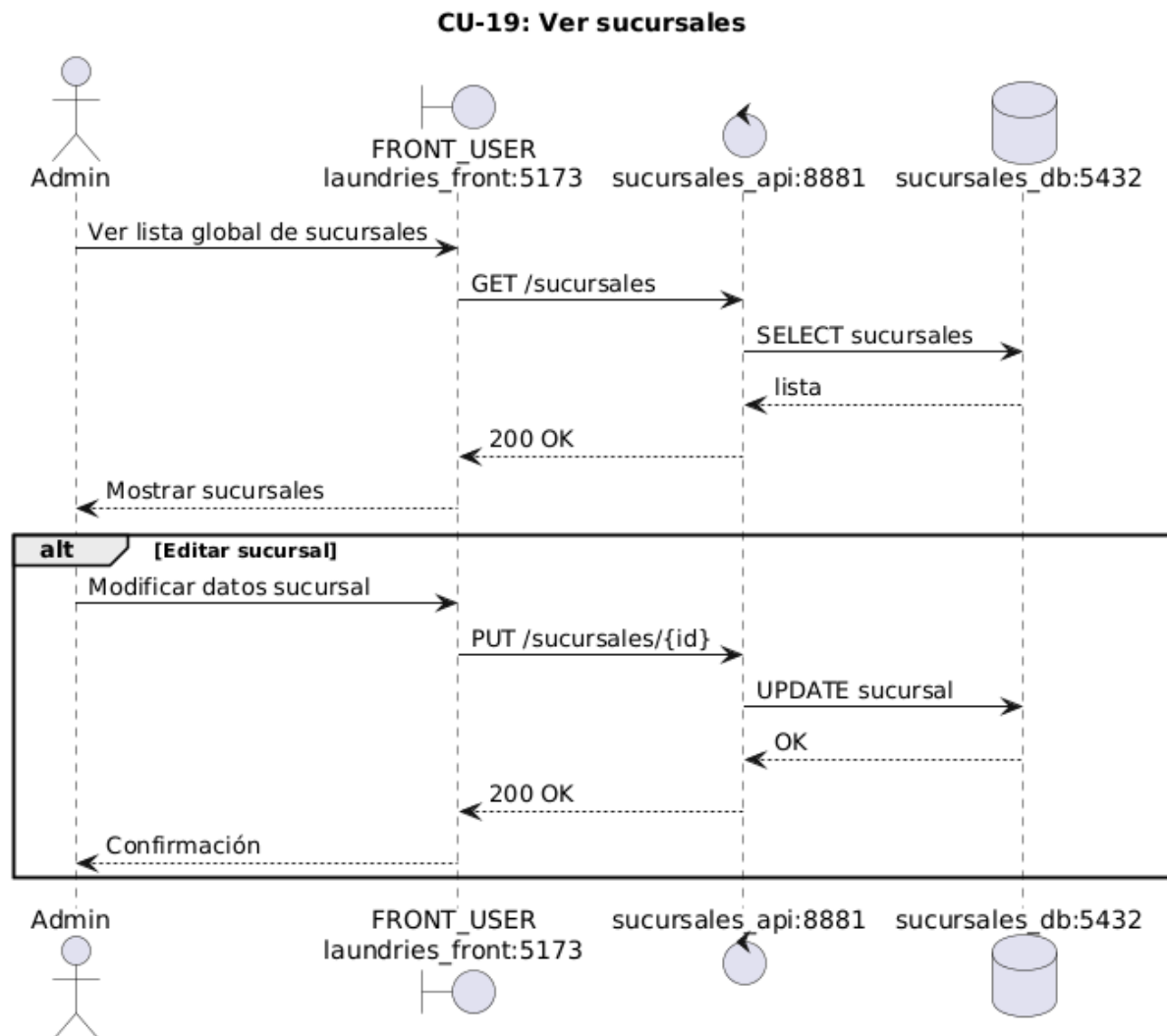
CU-16: Ver detalle de una venta activa.



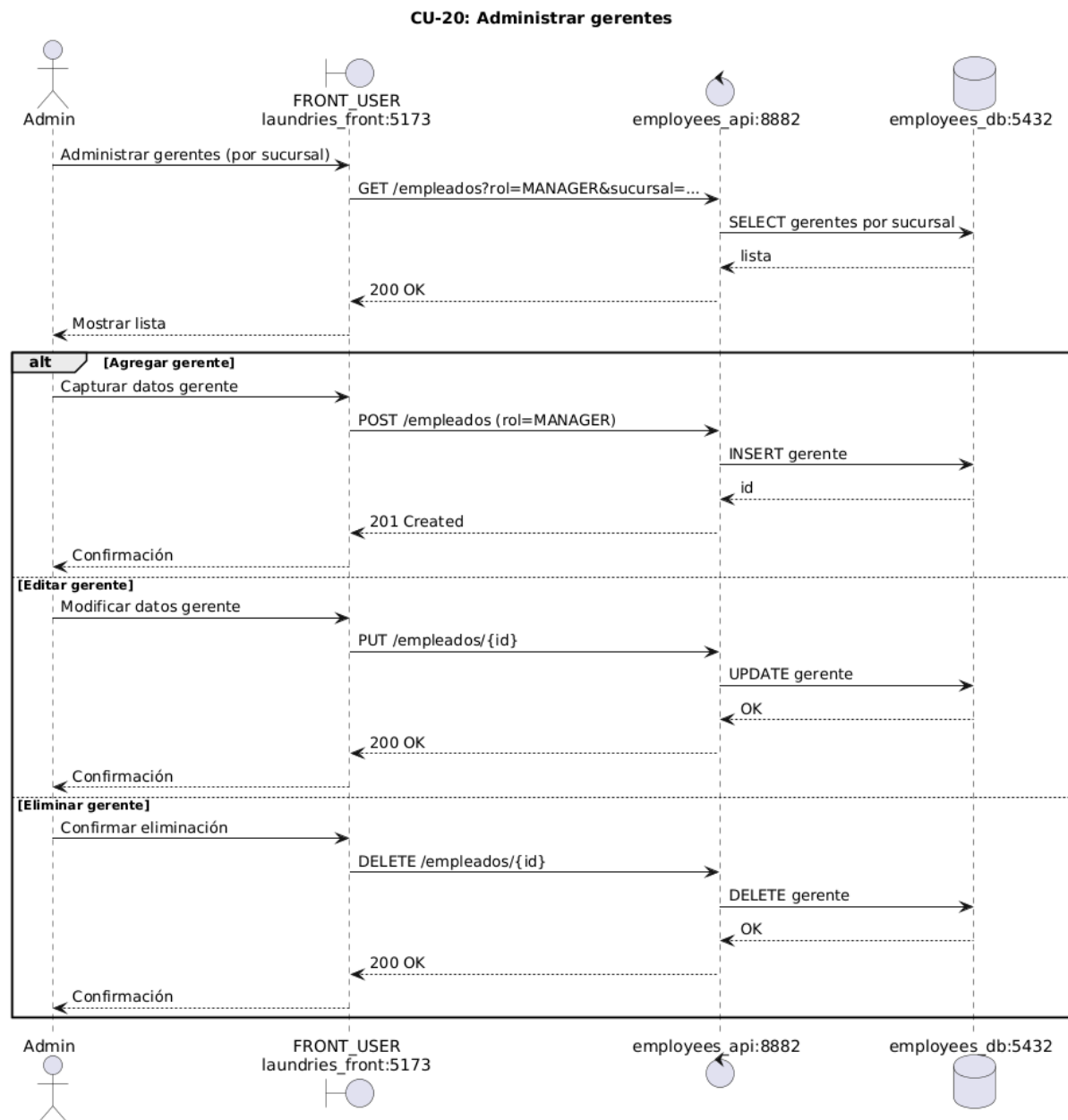
CU-17: Obtener código de cancelación.



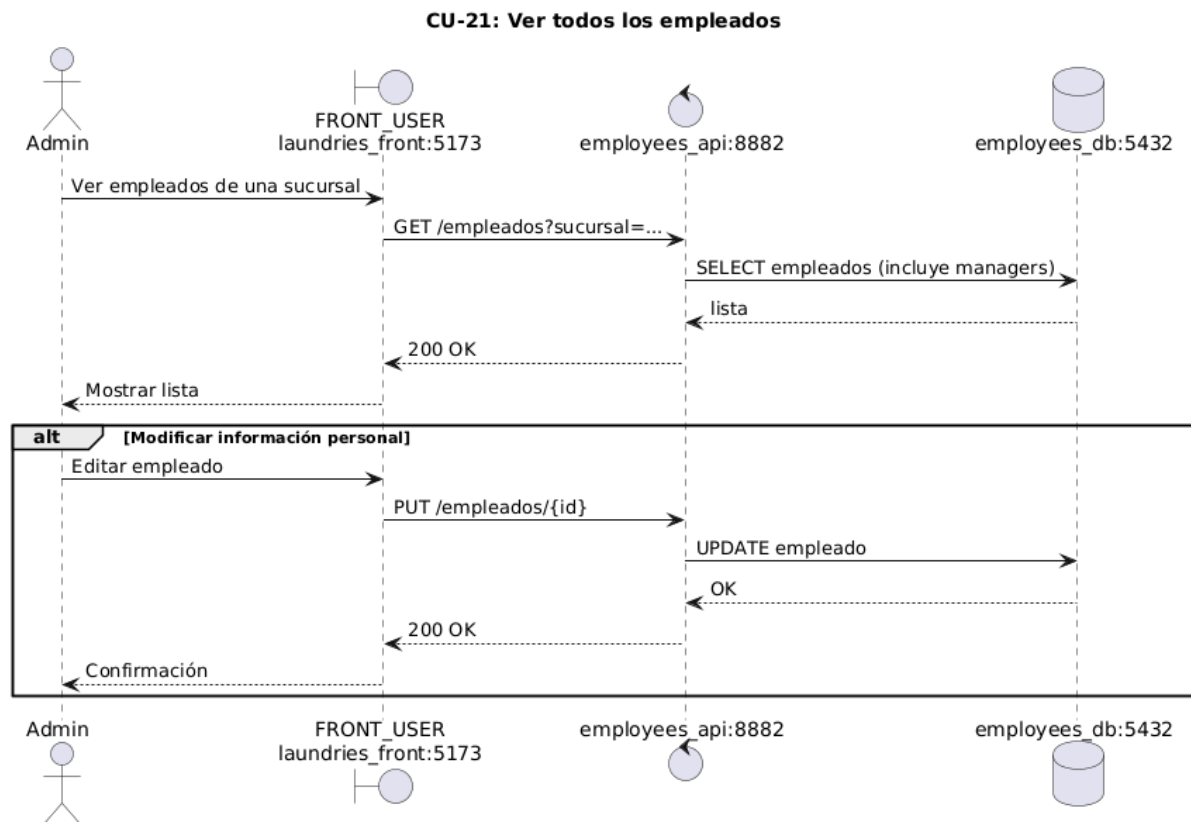
CU-19: Ver sucursales.



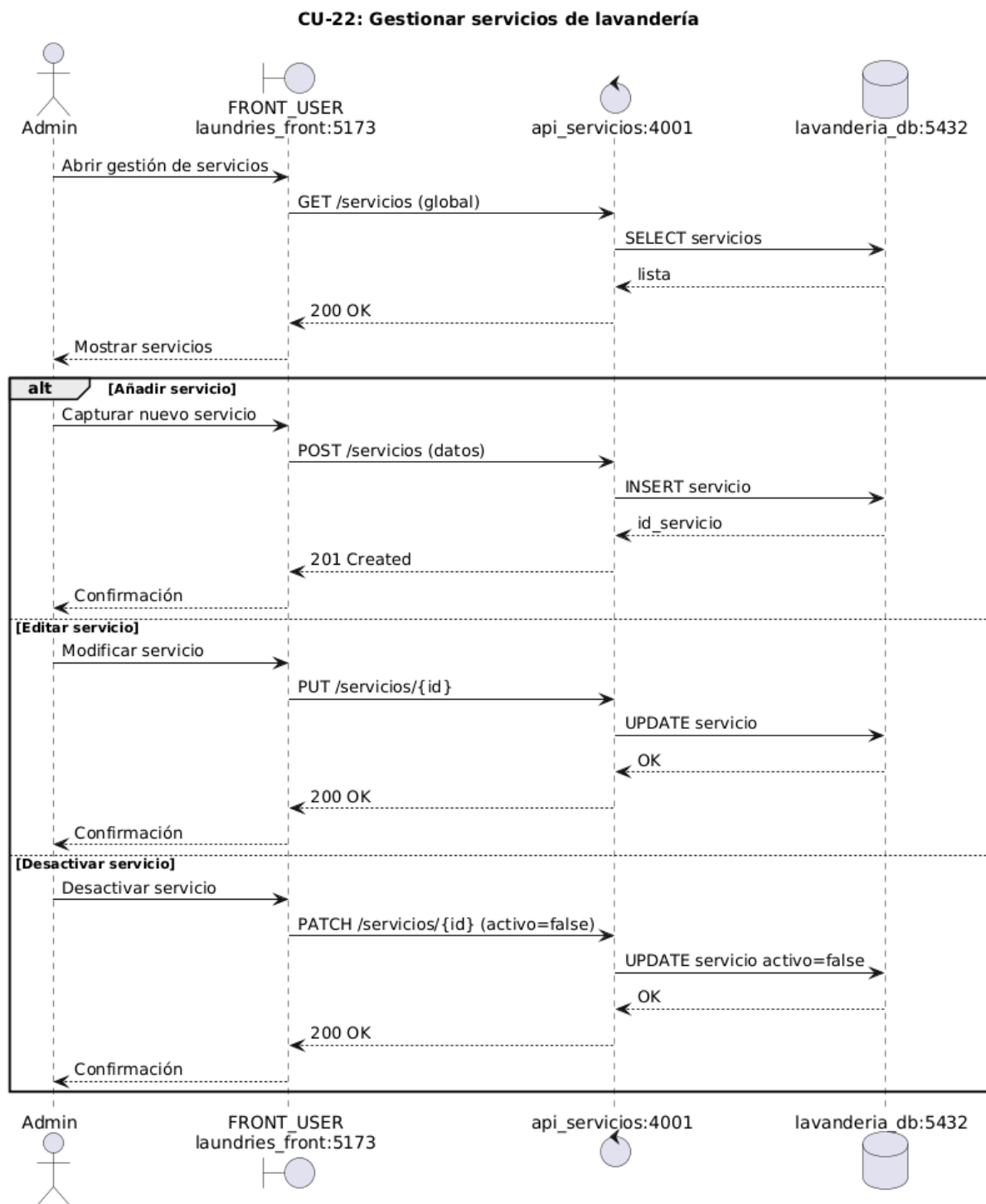
CU-20: Administrar gerentes.



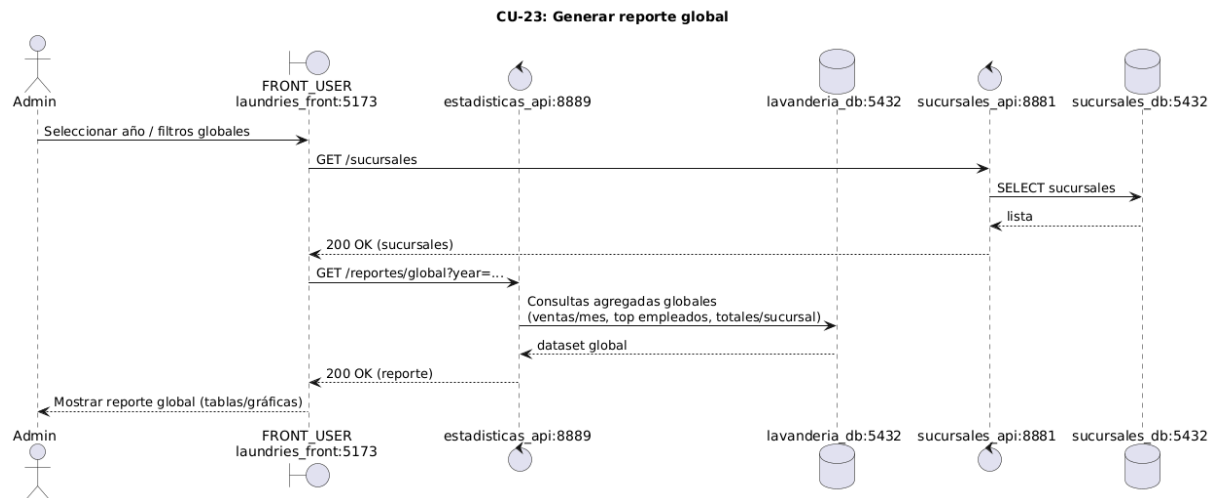
CU-21: Ver todos los empleados.



CU-22: Gestionar servicios de lavandería.



CU-23: Generar reporte global.



CU-24: Descargar reporte

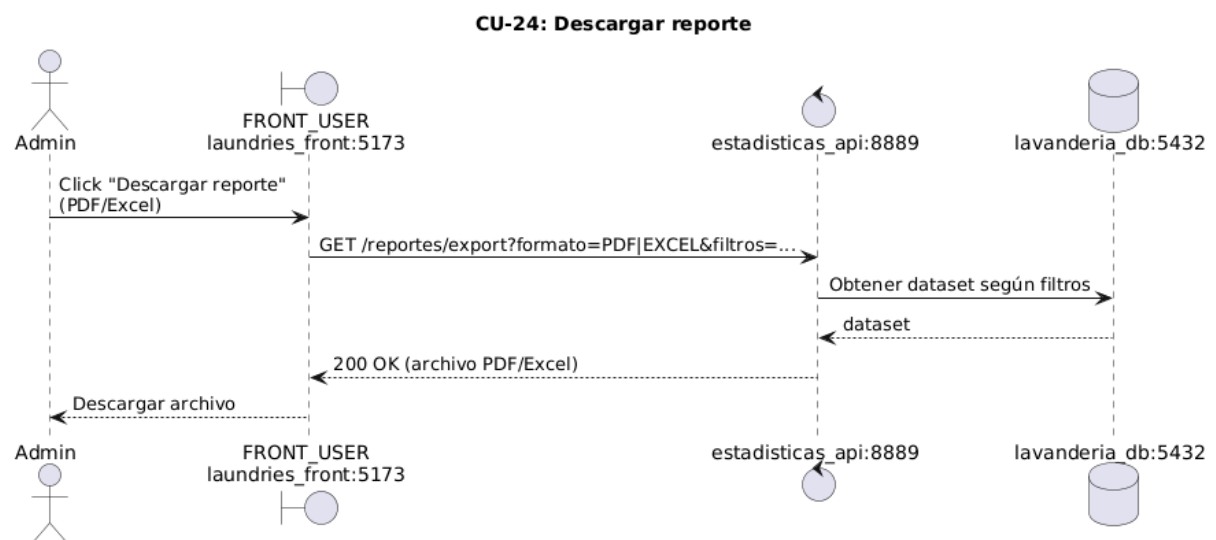


Diagrama de despliegue:

Los diagramas de despliegue son fundamentales en cualquier documentación de sistemas, estos muestran la organización funcional del sistema y la comunicación con las diferentes partes del mismo.

Para este proyecto específico, se ha utilizado una arquitectura de microservicios con un servidor de backend y otro servidor de frontend. Dado que se tiene un solo servidor de backend, aunque no es lo mejor, para las pruebas del proyecto se almacenaron todos los servicios dentro de un mismo servidor, aunque cada uno de ellos corre en su propio contenedor.

Cada API tiene su propia base de datos con su propia dirección y puerto. Estas APIs son consultadas por el navegador del cliente que está usando la aplicación a través de la página web. Cuando un usuario entra a la página, se le hace la solicitud a un servidor de frontend, que regresa la página y las demás solicitudes se hacen del navegador del cliente al servidor de backend, tal como muestra el diagrama de despliegue.

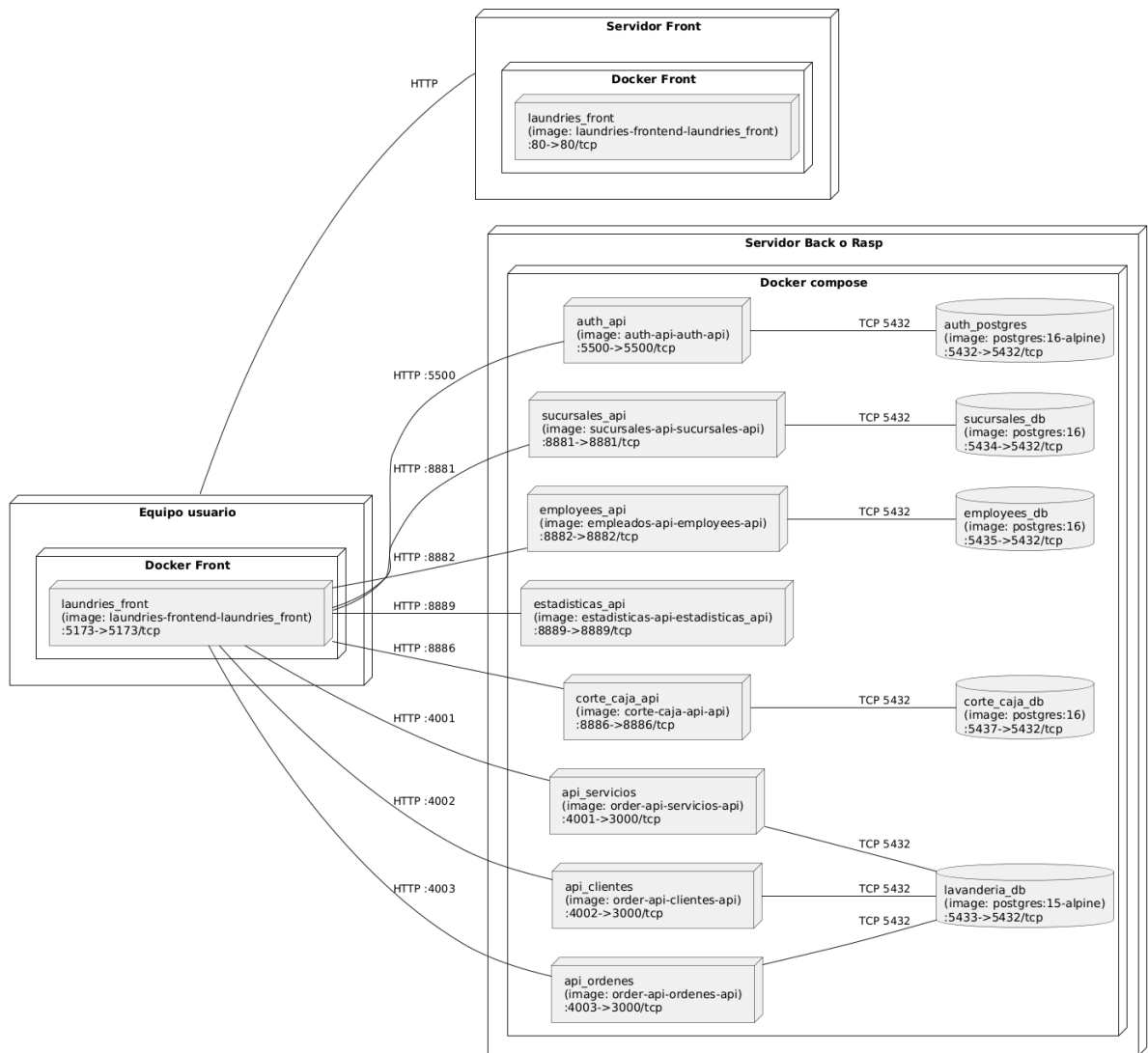


Diagrama entidad relación:

Los diagramas de entidad relación permiten conocer la forma en la que los datos se almacenan para su persistencia. Se decidió utilizar este tipo de diagramas para el sistema dado que documentan de buena manera los datos utilizados en el sistema. Cabe destacar que dado que la mayoría de APIs se encuentran como microservicios, los diagramas no se suelen relacionar entre sí, por lo que en muchos casos se encuentran como tablas sueltas.

A continuación se presentan los diagramas divididos según las APIS creadas.

Diagrama de orders-api:

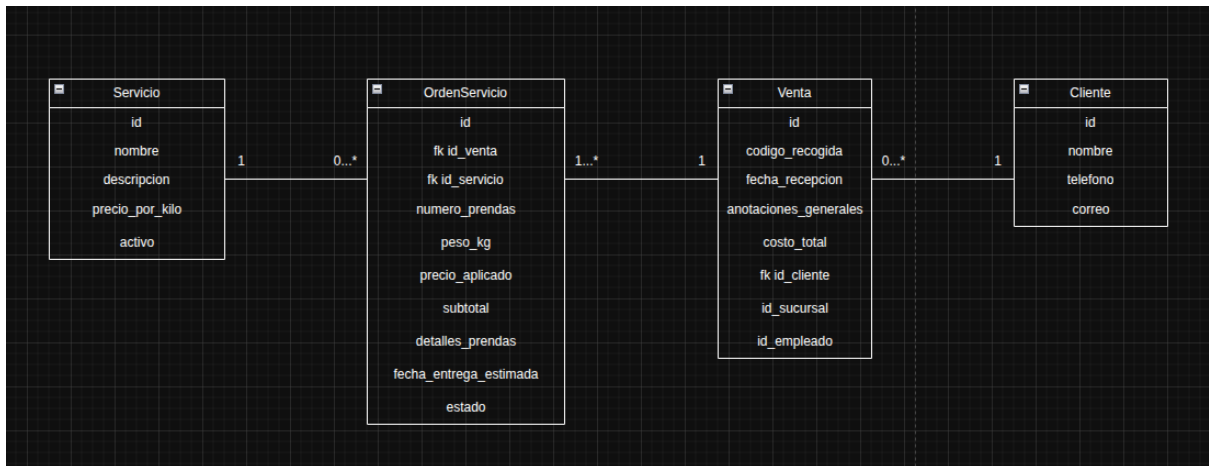


Diagrama de branches-api:

- Diagrama de empleados-api:

Empleado	
nombre	text
direccion	text
telefono	text
dni	text
fechaNacimiento	timestamp(3)
idSucursal	uuid
id	uuid

- Diagrama de sucursales-api:

sucursales	
nombre	varchar(100)
direccion	varchar(255)
telefono	varchar(30)
estado	boolean
clave_cancelacion	varchar(50)
id	uuid

- Diagrama de corte-caja-api:

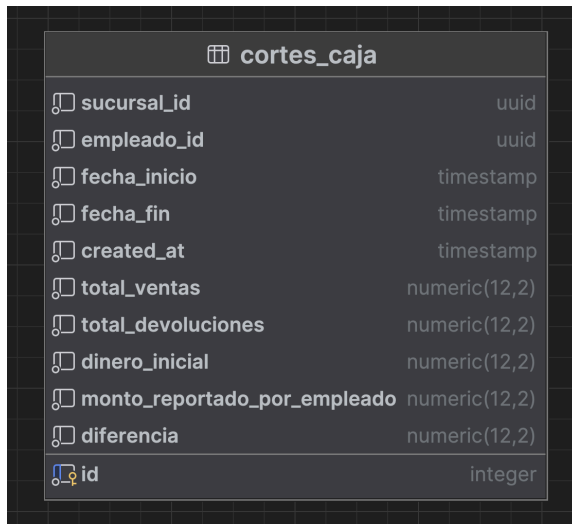
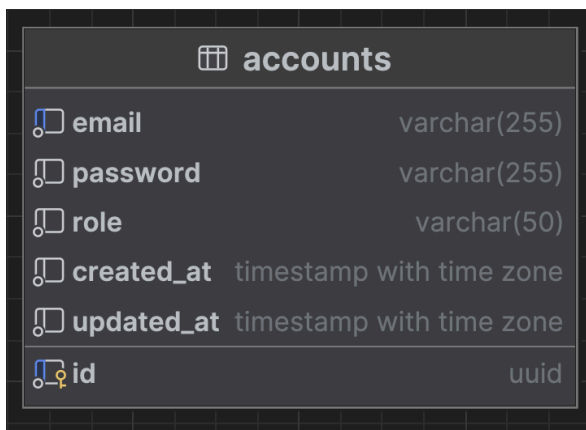


Diagrama de auth-api:



Casos de uso:

Con el objetivo de agilizar el desarrollo sin perder calidad, para este proyecto se ha propuesto la utilización de historias de usuario. A diferencia de las descripciones de casos de uso, las historias de usuario permiten tener un vistazo de lo que conlleva el caso de uso desde la perspectiva del usuario que lo realiza. Las historias de usuario se componen de tres partes, el usuario que la realiza expresado con

“Como”, la acción que desea realizar expresada como “Quiero”, y finalmente el objetivo que desea alcanzar expresado con la frase “Para”.

Gracias a las historias de usuario, se puede documentar el software de manera más ágil, sin perder calidad y siguiendo los estándares utilizados en la industria. Actualmente las historias de usuario son el método más utilizado para documentar dentro de la industria por sus enormes ventajas que ofrece, por lo que para este proyecto se decidió seguir este mismo enfoque.

Para dividir de manera lógica las historias de usuario, se decidió separarlas según el usuario que las realiza, teniendo cuatro secciones:

1. Los usuarios generales que incluye a los empleados, gerentes de las sucursales y administradores,
2. Los usuarios empleados,
3. Los usuarios gerentes de cada sucursal,
- y 4. Los usuarios administradores del sistema.

Historias de Usuario Generales: Employee, Manager y Admin

CU-01: Iniciar sesión

Yo como cualquier Usuario (Empleado, Gerente o Administrador), quiero poder iniciar sesión proporcionando mi correo electrónico y contraseña, para poder acceder a las funcionalidades correspondientes a mi rol y mantener mi información de trabajo segura.

CU-02: Cerrar sesión

Como cualquier Usuario (Empleado, Gerente o Administrador), quiero poder cerrar mi sesión de forma segura y fácil presionando el botón Logout, para evitar que otras personas puedan acceder a la información sensible de la sucursal o realizar cambios no autorizados.

Historias de Usuario: Employee (flujo diario)

CU-03: Ver cliente

Yo como Empleado quiero poder buscar rápidamente un cliente utilizando su nombre, teléfono o correo electrónico, para así poder agilizar el proceso de identificación del cliente en el mostrador.

CU-04: Buscar cliente

Yo como empleado quiero poder crear un cliente en dado caso de que no exista, primero verificando en la barra de búsqueda.

CU-05: Editar cliente

Yo como empleado quiero editar la información personal de un cliente ya sea porque me equivoqué al digitar o porque el cliente desea hacer un cambio en su información personal.

CU-06: Ver servicios activos

Yo como empleado quiero tener acceso a la lista de servicios activos a la hora de crear una venta , junto con sus precios activos para poder seleccionar las opciones correctas a la hora de recibir el pedido del cliente.

CU-07: Realizar venta

Yo como empleado quiero poder crear una nueva venta (ticket) para un cliente que llegue a la lavandería, ingresando varias órdenes o ítems con su respectivo servicio y peso, para que el sistema calcule el costo total de las subórdenes y la venta general de forma automática.

CU-08: Descargar comprobante

Yo como empleado quiero poder descargar el comprobante o ticket de la orden en formato de factura, el cual estará listo para ser impreso, justo después de crear la venta, para entregársela al cliente como constancia de su pedido y código de recogida.

CU-09: Modificar orden

Yo como empleado quiero poder modificar el estado de una prenda (ítems individuales) a LAVANDO, LISTO o ENTREGADO, para poder reflejar el progreso de las órdenes en tiempo real.

CU-10: Cancelar orden

Yo como empleado quiero poder solicitar CANCELAR una orden, para que al momento de cancelarla, el gerente de la sucursal introduzca su código secreto de cancelación, que corresponde al de una sucursal.

CU-11: Ver ventas activas

Yo como empleado quiero poder ver todas las ventas activas de mi sucursal, para poder ver mis tareas pendientes y priorizar la ropa que tenga que ser procesada.

CU-12: Ver ticket

Yo como empleado quiero poder tener acceso al ticket que fue generado al momento de crear la orden, y así poder descargarlo de nuevo no solo en el momento de la venta sino también en mi panel de órdenes pendientes.

CU-13: Realizar corte de caja

Yo como empleado quiero poder hacer un corte de caja todos los días al final del día para poder generar el monto resultante de las operaciones financieras del día respectivo.

Historias de Usuario: Manager (Supervisa y Controla)

CU-14: Gestionar empleados sucursal

Yo como gerente quiero poder gestionar a los empleados de mi sucursal, pudiendo ver la lista de los mismos, y también pudiendo agregarlos, eliminarlos o editarlos cuando lo requiera.

CU-15: Ver ventas activas de la sucursal

Yo como gerente, quiero poder ver todas las ventas activas de mi sucursal, con la opción de filtrar por algún cliente en específico y también para ver quién fue el que atendió al cliente, para supervisar el volumen de trabajo en tiempo real.

CU-16: Ver detalle de una venta activa

Yo como gerente quiero acceder al detalle completo de cualquier venta activa, incluyendo el detalle de todas las prendas, el costo total y las anotaciones, para poder resolver las dudas de algún cliente o empleado.

CU-17: Obtener código de cancelación

Yo como gerente quiero tener la capacidad de cancelar una orden en específico de un cliente, introduciendo de forma física el código de seguridad correspondiente a la sucursal.

CU-18: Generar reportes de sucursal

Yo como gerente quiero tener la capacidad de acceder a los reportes de la información de mi sucursal, pudiendo obtener información detallada en una fecha en específico de acuerdo a mis requerimientos específicos.

Historias de Usuario: Admin (General)

CU-19: Ver sucursales

Yo como administrador quiero poder tener acceso a la lista de mis sucursales de forma global, de modo que pueda editar la información de las mismas a mi preferencia.

CU-20: Administrar gerentes

Yo como administrador quiero poder administrar los gerentes de cualquier sucursal en específico, pudiendo agregarlos en los casos en los que desee hacerlo.

CU-21: Ver todos los empleados

Yo como administrador quiero poder ver a todos los empleados de cualquier sucursal en específico, incluyendo empleados comunes y gerentes, para poder modificar su información personal en los casos en los que lo requiera.

CU-22: Gestionar servicios de lavandería

Yo como administrador quiero poder ver todos los servicios que ofrecen las lavanderías de forma global, de modo que pueda gestionarlos añadiendo, editando o desactivando los servicios que estén presentes en el sistema.

CU-23: Generar reporte global

Yo como administrador quiero poder acceder a una sección de reportes en donde pueda ver el monto de dinero recaudado según las ventas por mes, donde también pueda ver los empleados que más ventas han realizado en el año, y por último, donde se pueda evidenciar el monto de dinero recaudado en el año seleccionado de acuerdo a cada sucursal.

CU-24: Descargar reporte

Yo como administrador quiero poder descargar un pdf o excel con la información respectiva a los reportes que son generados en el sistema para poder dar seguimiento a las ventas y estadísticas de todas mis lavanderías.

Construcción:

Selección justificada de la arquitectura:

La arquitectura del proyecto en general se basa en un cliente y en servidor, cada uno de estos con su arquitectura definida de la que se hablará más adelante. Cabe destacar que no existe un único servidor, se tiene un servidor de frontend y uno de backend, además que el de backend es de microservicios.

La decisión de separar el frontend del backend surgió a partir de la naturaleza de la web, lo cual permite consultar servicios externos fácilmente sin necesidad de almacenar información dentro del dispositivo. Además que, dado que el sistema compartirá datos constantemente y de manera real entre muchos usuarios, es mejor centralizarlos en un servidor al que consulten constantemente.

Pasando ahora a la arquitectura utilizada para el cliente, se utilizó un servidor que envía la página completa al cliente, además que esta consta de una sola página que se va modificando y cambiando de forma dinámica, lo que permite una mejor interacción y fluidez. Esta arquitectura es conocida como SPA, dado que solo tiene una página. La elección de este tipo de arquitectura se basó en el hecho de que es una aplicación interna de alta interacción. SPA tiene la ventaja de ser fluida en múltiples cambios e interacción, pero su desventaja es que su SEO es peor que en otras arquitecturas. Dado que la página es de uso interno, el SEO es poco relevante, pero como tiene una alta interacción y cambio de datos si se requiere que sea fluida, y es justo donde SPA destaca.

Adicionalmente se utilizó una arquitectura “Lazy” que mejora el rendimiento de la página, además de un router que va cambiando la página de forma quirúrgica, aumentando el rendimiento y la sensación de velocidad de la aplicación, lo cual fue específico para el tipo de sistema que se realizó.

La arquitectura de la aplicación fue FSD, la cual permite dividir el trabajo en casos de uso individuales, esto permitió al equipo desarrollar la

aplicación de forma ágil al estar en constante comunicación y trabajando con funcionalidades individuales sin generar conflictos dentro del repositorio usado.

Para el caso del servidor, se determinó que como se tienen muchos servicios independientes y se busca alta velocidad, se propuso el uso de microservicios. Esta arquitectura permite que se tenga un buen rendimiento y se desarrollen de manera más ágil, además que la información entre microservicios se comparte con llamadas a APIs, lo cuál permite compartir datos de forma segura y sencilla.

Cada microservicio utilizó su arquitectura propia, algunos de ellos utilizaron la arquitectura Clean o hexagonal, la cuál fue seleccionada para permitir una mejor escalabilidad y poder migrar el proyecto fácilmente si las tecnologías se desean cambiar en un futuro, lo cuál es preciso dado que el proyecto se encuentra en una etapa inicial. La arquitectura hexagonal separa la tecnología de la implementación, lo que permite migraciones rápidas y eficientes, para algunos microservicios como el de autenticación, el utilizar esta arquitectura en un estado puro permitió lograr una excelente calidad para el tipo de proyecto.

Cada microservicios tiene su propia base de datos y se comunican a través de peticiones HTTP, lo que permite compartir información de forma rápida, segura y sencilla.

Cabe destacar que los microservicios se encuentran diseñados para ser almacenados en diferentes servidores, pero dado que aún se está probando el proyecto en equipos de desarrollo pequeños, actualmente se cuenta con un solo servidor con arquitectura ARM64, lo cuál permite acceder al servidor desde cualquier lugar, sin embargo todos los servicios se encuentran almacenados en diferentes contenedores de docker que se comunican con otros contenedores con sus bases de datos específicos para cada uno de ellos.

Selección justificada de las tecnologías:

Las tecnologías empleadas para el proyecto fueron varias dependiendo de la parte a la que corresponden, pudiendo dividir principalmente en las orientadas al frontend, al backend y las del servidor, por lo que a continuación se hablará de cada stack tecnológico utilizado para las diferentes partes.

Para el frontend, dado que será una app web únicamente, se utilizó un stack orientado puramente al web, además que al ser de una arquitectura SPA, se eligió una tecnología que permitirá la construcción de este tipo de aplicaciones con calidad. Es por esta razón que se decidió utilizar REACT como tecnología para el frontend. Esta tecnología permite tener un SPA de manera sencilla y rápida, además que al utilizarse con SWC la compilación fue más rápida, aunado a esto, como empaquetador se utilizó Vite, lo que simplifica todas las operaciones de compilación para el frontend.

Como lenguaje para el cliente, se optó por utilizar TypeScript, dado que permite tener una mejor calidad en el código al utilizar tipos, y se considera una tecnología más novedosa recomendada para usar.

Para el caso del backend, las tecnologías fueron muy cambiantes según el API, sin embargo, en su mayoría se utilizó JavaScript como lenguaje con Node JS, esto permitió hacer el proyecto en un lenguaje similar al del frontend, lo que simplifica la integración del equipo y desarrollo, además que se utilizó como framework Express JS, lo que permitió utilizar APIS de forma rápida y con calidad. Estas APIS además se complementaron con un swagger que permitió su correcta documentación en la web.

Cabe destacar que algunas APIs se realizaron en el lenguaje de Python con el framework de Fast API, esto permitió hacer microservicios con características únicas, más rápidos y con una tecnología diferente más ágil, lo que a su vez logró que una parte del equipo de desarrollo lograra crear APIs y su documentación de manera más ágil que otros.

Como base de datos, se utilizó PostgreSQL, dado que en la industria se considera de buena calidad por su facilidad de uso, su seguridad y su

fácil integración con las tecnologías seleccionadas. Cada una de los microservicios tiene su propia base de datos, pero en general utilizan la misma tecnología.

Para las pruebas se utilizó Postman, lo cuál permite hacer solicitudes HTTP de manera sencilla y poder documentar los resultados de cada solicitud, lo que fue de gran utilidad para la creación de las pruebas documentadas.

Para el caso del servidor se utilizó una arquitectura basada en ARM, dado que se mantendría prendido durante mucho tiempo, esta arquitectura fue ideal por su bajo consumo energético. El sistema operativo utilizado fue Raspberry Pi Os Lite, este es un sistema operativo basado en Debian con funcionalidades mínimas y sin escritorio, el cuál fue accedido únicamente de manera remota mediante SSH. Dentro del sistema se utilizó Git y GitHub para almacenar y cambiar las versiones del servidor, además que, para que se accediera de forma privada y segura desde internet, se utilizó TailScale, una VPN que permite acceder a un servidor privado desde una cuenta compartida o con accesos a cuentas otorgados, lo que permite que el servidor no sea atacado por usuarios no autorizados.

Pruebas:

Las pruebas constituyen una parte elemental del sistema, estas garantizan que las funcionalidades básicas del mismo funcionen como se espera. A continuación se presenta una tabla que muestra todas las pruebas realizadas para este proyecto.

ID	Módulo	Caso de Uso	Método	Resultado Esperado	Resultado Obtenido	Estado
----	--------	-------------	--------	--------------------	--------------------	--------

CP-01	Autenticación	CU-01 Iniciar sesión	POST	Acceso exitoso según rol	Acceso concedido	 Exitoso
CP-02	Autenticación	CU-02 Cerrar sesión	POST	Sesión finalizada correctamente	Sesión cerrada	 Exitoso
CP-03	Clientes	CU-03 Ver cliente	GET	Datos del cliente devueltos	Datos correctos	 Exitoso
CP-04	Clientes	CU-04 Buscar cliente	GET	Cliente encontrado	Cliente localizado	 Exitoso
CP-05	Clientes	CU-04 Crear cliente	POST	Cliente creado	Cliente registrado	 Exitoso
CP-06	Clientes	CU-05 Editar cliente	PUT	Información actualizada	Cambios guardados	 Exitoso
CP-07	Empleados	Crear empleado	POST	Empleado creado	Registro exitoso	 Exitoso
CP-08	Empleados	Crear gerente	POST	Gerente creado	Registro exitoso	 Exitoso
CP-09	Empleados	Editar empleado	PUT	Datos actualizados	Cambios aplicados	 Exitoso

CP-10	Empleados	Editar gerente	PUT	Datos actualizados	Cambios aplicados	 Exitoso
CP-11	Empleados	Consultar empleado	GET	Información del empleado	Datos correctos	 Exitoso
CP-12	Empleados	Listar empleados	GET	Lista de empleados	Lista devuelta	 Exitoso
CP-13	Empleados	Empleados por sucursal	GET	Lista filtrada	Datos correctos	 Exitoso
CP-14	Sucursales	Crear sucursal	POST	Sucursal creada	Registro exitoso	 Exitoso
CP-15	Sucursales	Editar sucursal	PUT	Datos actualizados	Cambios aplicados	 Exitoso
CP-16	Sucursales	Consultar sucursal	GET	Información de sucursal	Datos correctos	 Exitoso
CP-17	Sucursales	Listar sucursales	GET	Lista de sucursales	Lista devuelta	 Exitoso
CP-18	Sucursales	Clave devolución	GET	Clave generada	Clave válida	 Exitoso
CP-19	Sucursales	Validar clave	POST	Clave válida	Validación correcta	 Exitoso

CP-20	Caja	Corte de caja	POST	Corte registrado	Corte creado	 Exitoso
CP-21	Caja	Reporte de cortes	GET	Reporte generado	Reporte correcto	 Exitoso
CP-22	Estadísticas	Ventas	GET	Estadísticas devueltas	Datos correctos	 Exitoso

Conclusiones:

Sin duda este proyecto ha sido de gran utilidad para el equipo de desarrollo, es innegable el hecho de que, la necesidad de escoger tecnologías web implicó una profunda investigación para conocer las ventajas que nos ofrecen cada una de ellas para este proyecto específico, además que la selección de la arquitectura fue clave para garantizar una buena calidad dentro del proyecto en un corto tiempo.

Es importante destacar que el uso de arquitecturas de alta calidad como la hexagonal requirió que el equipo pasara por una curva de aprendizaje muy elevada, sin embargo esto les aportó a cada uno de ellos la experiencia necesaria para trabajar en estas arquitecturas en un entorno real.

Adicionalmente, el trabajar con un equipo internacional les dió la experiencia dentro del equipo de compartir formas de organización y nuevos métodos empleados por diferentes países, lo que sin duda fue muy enriquecedor.

Sin duda este proyecto ha aportado experiencia y una base de nuevos conocimientos a todo el equipo de desarrollo, lo cuál es de gran utilidad en su formación como ingenieros de software.